

Laboratory 1: Short project

Docker, Compose and Swarm

Member 1

Member 2

Member 3

Member 4

Member 5

Cloud Computing and Distributed Systems

MSc in Machine Learning and Cybersecurity for Internet Connected Systems, UPC

October 2026

1 Introduction

We containerised the infrastructure of Figure 1 of the statement: the shared services (*basic*), a company application (*app*), an observability platform (*stats*) and Moodle (*atenea*). As the core application we wrote a small review site in Flask: people leave a comment with a rating from 1 to 5, the average is cached in Redis, each new comment sends a mail through our SMTP relay and an automatic feeder collects ratings published in other web pages.

The repository contains `compose.yml` (local), `production.yml` (swarm), one folder per image with its Dockerfile and files, and this document.

2 Architecture

Table 1 maps the boxes of the diagram to our services. Each stack is an overlay network, and a service only joins the networks it needs. For example the databases are not in the **proxy** network, so the frontend can not reach them.

Stack	Service	Description
basic	frontend	nginx, the only public service (80/443), proxies by subdomain
	letsencrypt	certbot, gets and renews one certificate for all subdomains
	registry	Docker registry with httpasswd authentication
	NFS	NFSv4 server (port 2049), shared disk for the nodes
app	core	Flask + gunicorn application
	static	Apache serving <code>/static/</code>
	db	PostgreSQL 16 (databases app and grafana)
	redis	cache of the average rating
	smtp	Postfix relaying the mails to an external provider
	auto	feeder: reads schema.org ratings of web pages
atenea	backup	cron that dumps the databases and moodledata
	moodle, moodle-cron	Moodle 4.5 LTS (web and cron, same image)
	moodle-db	MariaDB 11.4
stats	grafana	dashboards, with the Infinity plugin (the “?”)
	prometheus, cadvisor	metrics of the containers and the core app

Table 1: Services of the infrastructure.

Public names in production: `app.<domain>`, `lms.<domain>`, `grafana.<domain>` and `registry.<domain>`. Locally the same names are used with `localhost` on port 8080 (browsers resolve `*.localhost` to `127.0.0.1`).

3 Images and design decisions

Frontend. Based on `nginx:alpine`. We use the template feature of the official image (`envsubst` on startup) with two templates, local (`http`) and production (`https`), selected with `NGINX_ENVSUBST_TEMPLATE_DIR`. Upstreams are resolved at request time through the Docker DNS, so `nginx` starts even if a backend is not running yet. `nginx` refuses to start without a certificate, so a small script creates a self signed one and switches to the Let's Encrypt one as soon as `certbot` gets it.

Let's Encrypt. The official `certbot` image with a loop: `certbot certonly --webroot --keep-until-expiring` every 12 hours, which issues the certificate the first time and renews it later. The challenge files are written in a volume shared with `nginx`.

Core app. Python 3.12 slim, runs as a non-root user. It creates its tables on the first request, so there is no separate migration step. It exposes `/metrics` for Prometheus (blocked from outside in `nginx`) and `/api/stats` with JSON for Grafana. The page includes the average rating as JSON-LD (`schema.org`), the same format online shops use for search engines.

Auto. The feeder downloads the pages in `FEED_URLS`, extracts the `aggregateRating` of the JSON-LD and posts it to `/api/ratings`, protected by a bearer token. By default it reads our own app, but any page with that markup works.

SMTP. Postfix on Alpine. It only accepts mail from the Docker networks and relays it to the provider configured in `.env` with SASL authentication and mandatory TLS. In the local environment it relays to `mailpit`, so the mails can be checked in the browser.

Moodle. `php:8.3-apache` with the extensions Moodle needs and the 4.5 LTS release downloaded at build time. `config.php` reads the URL from the environment and the password from a Docker secret. The entrypoint waits for MariaDB and installs Moodle from the CLI the first time; to avoid two replicas installing at the same time, the one that manages to create a lock directory in the shared `moodledata` does it and the rest wait. `moodle-cron` is the same image running `admin/cli/cron.php` every minute.

Grafana. Configured as code: the datasources, the dashboard provider and the dashboard (JSON) are copied into the image, so the whole configuration is versioned in git. For the “?” we chose the **Infinity** datasource plugin, which turns any JSON, CSV or XML endpoint into a datasource. We use it to show the average rating and the ratings captured by the feeder directly from our API, without writing an exporter.

4 Deployment

4.1 Cluster

We used three Ubuntu 24.04 virtual machines in the same private network, all of them swarm managers so the cluster keeps working if one node fails (quorum of 2 out of 3). Node 1 has the label `db=true` and also runs the NFS server. A wildcard DNS record `*.<domain>` points to the nodes; thanks to the routing mesh any node accepts the traffic and forwards it to a frontend replica.

```
docker swarm init --advertise-addr 10.0.0.4 # node1, then join node2 and node3
docker node update --label-add db=true node1
```

4.2 Shared storage

The NFS server needs a privileged container and swarm services can not be privileged, so it is the only service started with plain Compose on the storage node (`basic/nfs/nfs.yml`). It exports `/srv/nfs` with NFSv4 only, restricted to the cluster subnet. In `production.yml` the shared volumes use the `local` driver with `type: nfs`, so Docker mounts them automatically on whatever node a task is scheduled. Nothing is mounted by hand on the hosts.

4.3 Compose files

- `compose.yml`: local environment, no TLS, named volumes only, mailpit instead of a real mail provider.
- `production.yml`: the whole infrastructure for the swarm. It keeps the `build:` keys, so the same file is used to build and push the images (`docker stack deploy` ignores them).

4.4 Steps

```
set -a; . ./env; set +a # docker stack does not read .env
docker compose -f production.yml build
docker compose -f production.yml push
docker stack deploy --with-registry-auth -c production.yml lab1
```

The images are pulled from our registry at `127.0.0.1:5000`. This address works on every node because the registry port is published on the routing mesh, and Docker accepts plain HTTP for localhost. On the very first deployment the registry does not exist yet, so we deploy once (the registry starts and the other services wait), push the images and deploy again. `--with-registry-auth` forwards the registry login to the other nodes.

5 Scalability

No service uses `container_name`, fixed IPs or host ports other than the routing mesh, so all of them can run with several containers. Table 2 shows the replicas we use and why some are kept at one.

Service	Replicas	Reason
core	3	stateless, data in PostgreSQL and Redis
frontend, static, smtp	2	stateless
registry	2	storage on NFS, same <code>REGISTRY_HTTP_SECRET</code>
moodle	2	moodledata on NFS, sessions in the database
grafana	2	its state is in PostgreSQL, not in a local SQLite
cadvisor	global	one per node
db, moodle-db	1	DBMS without automatic scaling
redis, prometheus	1	in-memory cache / local time series database
letsencrypt, backup, moodle-cron, auto	1	periodic jobs

Table 2: Replicas per service.

Updates use `order: start-first` and `failure_action: rollback`, so a new version only replaces the old one if it starts correctly. Prometheus discovers the replicas of the core app with the DNS name `tasks.core`, so after `docker service scale lab1_core=5` the new containers appear in Grafana without changes. The footer of the app shows the container that answered, which makes the load balancing easy to see.

Data	Volume	Location
PostgreSQL, MariaDB, Prometheus	db, moodle-db, prometheus	local disk of node 1
moodledata	moodledata	NFS
certificates, ACME challenges	certs, acme	NFS
registry images	registry	NFS
backups	backups	NFS
Redis	none	memory only (cache)

Table 3: Where the data is stored.

6 Data management and backups

We decided not to put the databases on NFS: database engines on network filesystems risk corruption with locking problems or network cuts, and they are slow. They are pinned with a placement constraint to the node labelled `db=true` and use local volumes; what goes to the shared disk are the dumps.

The backup service is an Alpine image with `crond`. Every night at 03:30 it creates a folder in the backup volume with:

- `app.dump` and `grafana.dump`: `pg_dump -Fc` (custom format, allows restoring single tables);
- `moodle.sql.gz`: `mariadb-dump --single-transaction`, consistent without locking the tables;
- `moodledata.tar.gz`: moodledata without caches, sessions and temporary files.

Folders older than 14 days are deleted. A backup can be launched by hand with `docker exec <backup container> /backup.sh`. Restoring, from node 1:

```
docker exec -i <db container> pg_restore -U app -d app --clean < /srv/nfs/backups/<date>/app.dump
gunzip -c /srv/nfs/backups/<date>/moodle.sql.gz | docker exec -i <moodle-db container> \
sh -c 'mariadb -u moodle -p"$(cat /run/secrets/moodle_db_password)" moodle'
tar -xzf /srv/nfs/backups/<date>/moodledata.tar.gz -C /srv/nfs/moodledata
```

The main limitation is that the backups are on the same server as the shared data. For a real deployment we would copy them off-site (for example with `restic` to object storage) and encrypt them. Also, if node 1 fails, the databases stop until they are restored on another node from the last dump.

7 Security

- Only `nginx` is exposed; port 80 only serves ACME challenges and redirects to HTTPS (TLS 1.2/1.3 and HSTS).
- Passwords are Docker secrets mounted in `/run/secrets`, never written in the compose files; the `secrets/` folder is ignored by git.
- The overlay networks `proxy`, `app` and `atenea` are encrypted (IPsec between nodes), and each stack is isolated in its own network.
- The SMTP relay is not published and only accepts mail from the internal networks.
- The registry requires authentication (bcrypt `htpasswd`) and `/metrics` is not reachable from outside.

Known weak points: ports published by Docker skip the host firewall (`ufw`), so the registry port 5000 must be closed in the cloud provider firewall; and the NFS export uses `no_root_squash` because some containers write as root, which we limited to the cluster subnet.

8 Conclusions

The whole infrastructure is deployed with one command per step (build, push, deploy) and every service can run on any node, except the databases, which we pinned on purpose. The part that took the most time was Moodle (installation with several replicas and the shared moodledata) and making nginx start before the real certificate existed.

Work distribution

- Member 1 basic stack: frontend, Let's Encrypt, registry, NFS
- Member 2 core app, static files, Redis
- Member 3 databases, SMTP relay, feeder, backups
- Member 4 Atenea (Moodle and MariaDB)
- Member 5 stats stack and swarm cluster